

Lab: File Path Traversal, Validation of File Extension with Null Byte Bypass

PortSwigger Web Security Academy — Parcial I: Fundamentos de Ciberseguridad

1. Nombre del laboratorio

File path traversal, validation of file extension with null byte bypass

Fuente oficial:

<https://portswigger.net/web-security/file-path-traversal/lab-validate-file-extension-null-byte-bypass>

2. Nivel

Practitioner.

3. Tipo de explotación

Path Traversal / Directory Traversal (CWE-22), con bypass de una validación de extensión de archivo ("ends with") mediante la inyección de un byte nulo (%00) que trunca la cadena a nivel del sistema de archivos mientras la validación de la aplicación sigue viendo la extensión esperada.

4. Objetivo del laboratorio

La aplicación vulnerable sirve imágenes de producto a través de GET /image?filename=NOMBRE. El servidor valida que el valor de filename termine con la extensión esperada (por ejemplo, ".jpg"). El objetivo es construir un payload que contenga una secuencia de traversal hacia /etc/passwd y, al mismo tiempo, termine formalmente en ".jpg" para satisfacer esa validación de sufijo.

5. Explicación técnica de la vulnerabilidad

5.1 Concepto: validar el final de la cadena, no el archivo real que se abre

La aplicación implementa un control de seguridad equivalente a `filename.endsWith(".jpg")`: si el valor recibido termina con la extensión esperada, se asume que es una imagen legítima y se usa directamente para localizar y leer el archivo. Este control no analiza el resto de la cadena ni comprueba que el archivo resultante sea, en la práctica, una imagen dentro del directorio permitido.

El punto débil es que existe un desajuste entre dos capas del sistema: la capa de aplicación (donde corre la validación, normalmente en un lenguaje de alto nivel) procesa la cadena completa, carácter por carácter, incluyendo cualquier byte que contenga; en cambio, muchas rutinas de bajo nivel del sistema operativo y de bibliotecas nativas de manejo de cadenas (basadas en el estilo de C, donde una cadena termina en el primer byte nulo, 0x00) se detienen abruptamente al encontrar ese byte. Si el valor decodificado llega con un byte nulo incrustado, la aplicación "ve" toda la cadena (incluida la extensión al final) para efectos de validación, pero el sistema de archivos, al abrir el archivo, corta la cadena justo en el byte nulo e ignora todo lo que viene después.

5.2 Por qué falla el traversal simple

Al enviar `../../../../etc/passwd` directamente, la cadena no termina en `".jpg"`; la validación de extensión la rechaza de inmediato (400 Bad Request, "No such file").

5.3 El bypass: byte nulo antes de la extensión

La clave es construir la ruta de traversal real y, después de ella, agregar un byte nulo codificado en URL (`%00`) seguido de la extensión esperada:

- Payload enviado: `../../../../etc/passwd%00.jpg`
- El framework decodifica `%00` a un byte nulo real dentro del valor de filename. Para la lógica de validación de la aplicación, la cadena completa (`"../../../../etc/passwd\0.jpg"`) efectivamente termina en `".jpg"`, así que la comprobación de extensión se satisface sin problema.
- Cuando ese valor se usa para localizar el archivo en el sistema de archivos (a través de una función o API que trata la cadena al estilo C, terminada en el primer byte nulo), todo lo que viene después del byte nulo —incluida la extensión `".jpg"`— se descarta.
- Lo que el sistema operativo realmente intenta abrir es `../../../../etc/passwd`, sin ninguna extensión, que es exactamente el archivo objetivo.
- El servidor responde 200 OK con el contenido completo de `/etc/passwd`.

5.4 Impacto y clasificación estándar

El impacto es el mismo de toda la familia de laboratorios de path traversal: exposición de información sensible del sistema (confidencialidad), clasificada como CWE-22 (Improper Limitation of a Pathname to a Restricted Directory), con una causa raíz específica que suele documentarse aparte como CWE-158 (Improper Neutralization of Null Byte or NUL Character), y encuadrada en OWASP Top 10 2021 — A01:2021 Broken Access Control. Este es un patrón de vulnerabilidad históricamente asociado a la interoperabilidad entre lenguajes gestionados (que permiten bytes nulos dentro de cadenas) y bibliotecas nativas subyacentes (que no los permiten); muchas plataformas modernas ya mitigan este problema de fábrica rechazando bytes nulos en rutas de archivo, pero sigue siendo un patrón de fallo importante de reconocer en código legado o en integraciones de bajo nivel.

La defensa efectiva combina rechazar de forma explícita cualquier entrada que contenga bytes nulos u otros caracteres de control, validar la ruta canonicalizada resultante en lugar de solo el sufijo de la cadena original, y aplicar el principio de menor privilegio como capa adicional (ver sección 8, Mitigación recomendada).

6. Procedimiento paso a paso

Paso 1 — Reconocimiento del laboratorio

Se accede a la descripción oficial del laboratorio, confirmando el nivel Practitioner: la aplicación valida que el filename proporcionado termine con la extensión de archivo esperada.

Lab: File path traversal, validation of file extension with null byte bypass

PRACTITIONER



This lab contains a path traversal vulnerability in the display of product images.

The application validates that the supplied filename ends with the expected file extension.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



ACCESS THE LAB

Figura 1. Descripción oficial del laboratorio en PortSwigger Web Security Academy.

Paso 2 — Exploración del catálogo

Se navega por el catálogo de productos de la tienda, confirmando el estado "Not solved" antes de iniciar la explotación.



File path traversal, validation of file extension with null byte bypass

LAB Not solved

[Back to lab description >>](#)

[Home](#)

WE LIKE TO
SHOP

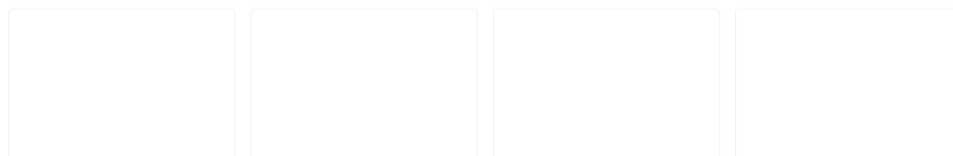


Figura 2. Catálogo de productos con el laboratorio en estado "Not solved".

Paso 3 — Localización del endpoint vulnerable

En el historial de peticiones (Proxy > HTTP history) se localizan las solicitudes GET `/image?filename=X.jpg` correspondientes a las imágenes de los productos.

Time	Type	Direction	Method	URL
23:09:35 28 Se...	HTTP	→ Request	GET	https://0a5a003303fa68c880143a0f009a004e.web-security-academy.net/image?filename=21.jpg
23:09:35 28 Se...	HTTP	→ Request	GET	https://0a5a003303fa68c880143a0f009a004e.web-security-academy.net/image?filename=11.jpg
23:09:35 28 Se...	HTTP	→ Request	GET	https://0a5a003303fa68c880143a0f009a004e.web-security-academy.net/image?filename=73.jpg
23:09:35 28 Se...	HTTP	→ Request	GET	https://0a5a003303fa68c880143a0f009a004e.web-security-academy.net/academyLabHeader
23:09:59 28 Se...	HTTP	→ Request	GET	https://0a5a003303fa68c880143a0f009a004e.web-security-academy.net/image?filename=37.jpg
23:09:59 28 Se...	HTTP	→ Request	GET	https://0a5a003303fa68c880143a0f009a004e.web-security-academy.net/image?filename=66.jpg
23:09:59 28 Se...	HTTP	→ Request	GET	https://0a5a003303fa68c880143a0f009a004e.web-security-academy.net/image?filename=8.jpg
23:09:59 28 Se...	HTTP	→ Request	GET	https://0a5a003303fa68c880143a0f009a004e.web-security-academy.net/image?filename=72.jpg

Request

Pretty Raw Hex

```

1 GET /image?filename=21.jpg HTTP/2
2 Host: 0a5a003303fa68c880143a0f009a004e.web-security-academy.net
3 Cookie: session=7hgkzF0UyYw0003eX3eRfetu96w2mJl
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-419,es;q=0.9
6 Sec-Ch-Ua: "Chromium",v="151", "Not=A?Brand",v="55"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0a5a003303fa68c880143a0f009a004e.web-security-academy.net/
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16

```

Figura 3. Historial de peticiones mostrando solicitudes al endpoint /image?filename=.

Paso 4 — Envío de la solicitud a Repeater

Se selecciona una de las solicitudes de imagen y, mediante el menú contextual, se envía al módulo Repeater para manipular el parámetro filename de forma controlada.

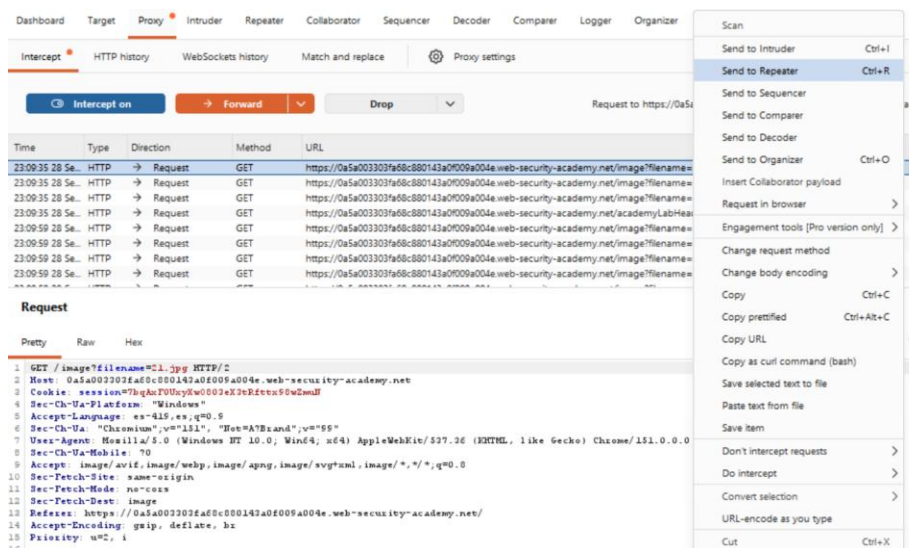


Figura 4. Envío de la solicitud interceptada al módulo Repeater ("Send to Repeater").

Paso 5 — Solicitud base en Repeater

Se confirma en Repeater la solicitud original sin modificar, como punto de partida antes de probar los distintos payloads.

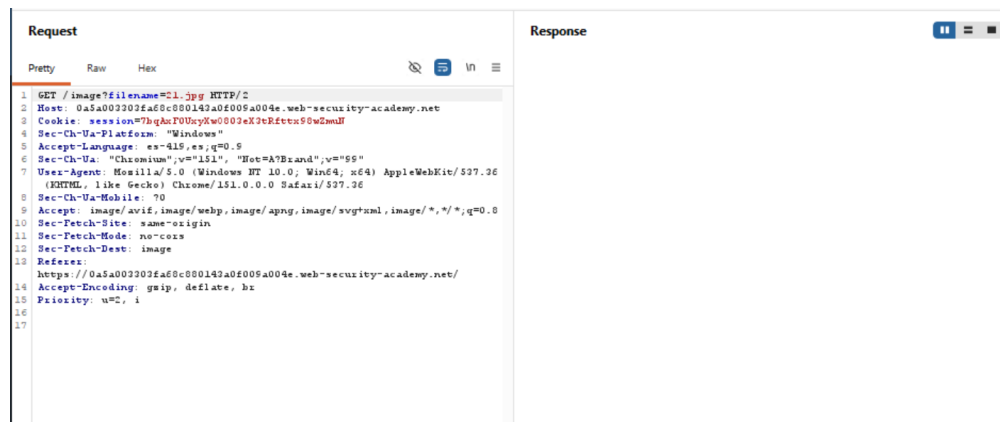


Figura 5. Solicitud original en el módulo Repeater, sin modificar.

Paso 6 — Prueba de traversal simple (bloqueada)

Se prueba el payload `../../../../etc/passwd`, sin la extensión esperada. El servidor responde 400 Bad Request con "No such file", confirmando que la validación de extensión rechaza cualquier valor que no termine en ".jpg".

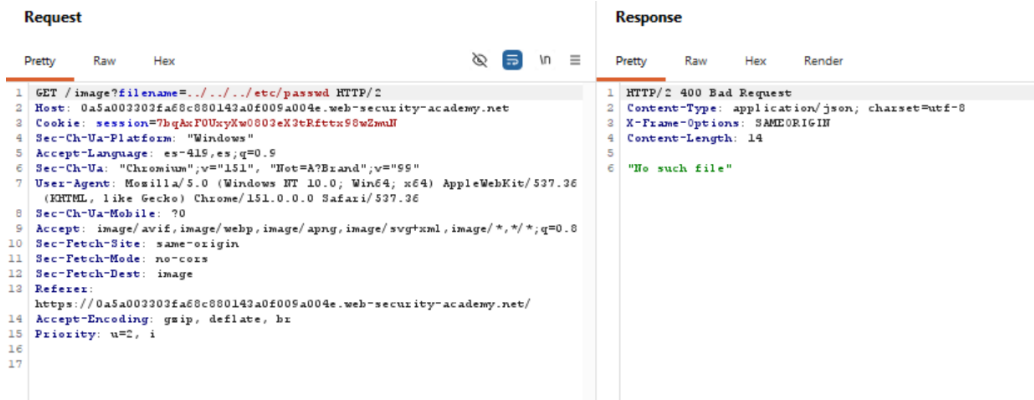


Figura 6. Payload `../../../../etc/passwd` bloqueado por la validación de extensión (400 Bad Request, "No such file").

Paso 7 — Bypass con byte nulo

Se sustituye filename por `../../../../etc/passwd%00.jpg`. La validación de extensión ve la cadena completa terminando en ".jpg" y la deja pasar; el sistema de archivos, al procesar la cadena, se detiene en el byte nulo y abre efectivamente `/etc/passwd`. El servidor responde 200 OK con el contenido completo del archivo.

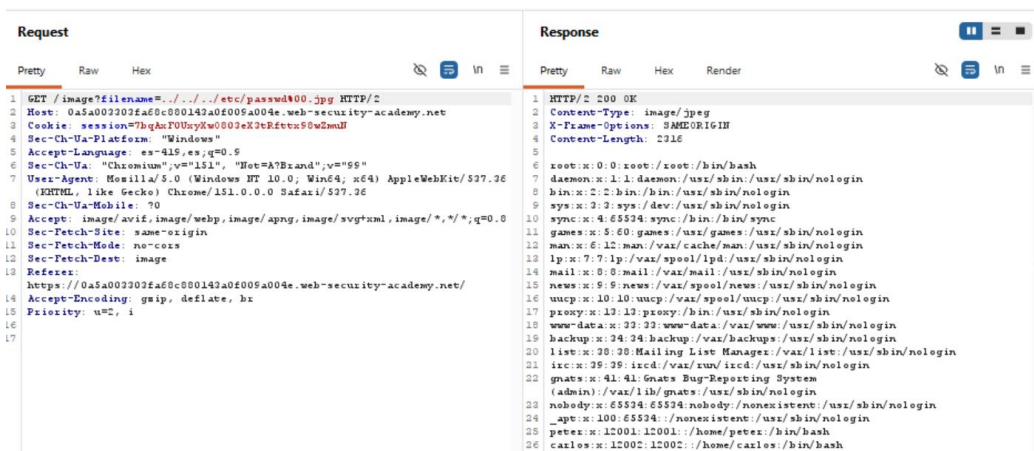


Figura 7. Payload `../../../../etc/passwd%00.jpg` — respuesta 200 OK con el contenido del archivo.

Paso 8 — Confirmación de resolución del laboratorio

PortSwigger Web Security Academy confirma la explotación exitosa marcando el laboratorio como "Solved".

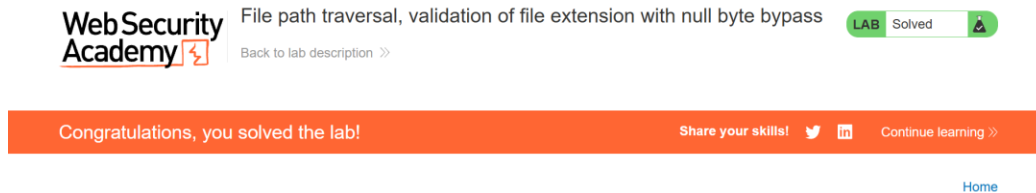


Figura 8. Confirmación "Lab Solved" en PortSwigger Web Security Academy.

7. Explicación del payload

Payload bloqueado:

`../../../../etc/passwd`

Payload exitoso:

`../../../../etc/passwd%00.jpg`

Funcionamiento técnico:

- El primer payload no termina en ".jpg", por lo que la validación de extensión lo rechaza antes de cualquier acceso al sistema de archivos.
- El segundo payload agrega, después de la ruta de traversal real, la secuencia %00 (el byte nulo codificado en URL) seguida de ".jpg".
- El framework decodifica %00 a un byte nulo real (0x00) dentro del valor de filename, antes de que la aplicación lo procese.
- La lógica de validación de extensión, escrita en un lenguaje de alto nivel, examina la cadena completa como una secuencia de caracteres (incluido el byte nulo) y confirma que termina en ".jpg" — la validación se satisface.
- Al usar ese mismo valor para abrir el archivo, la función o API subyacente (con comportamiento de cadena estilo C) interpreta el byte nulo como el final de la cadena, descartando todo lo que sigue. El archivo que realmente se intenta abrir es ../../../../etc/passwd, sin ninguna extensión.
- El sistema operativo resuelve esa ruta de traversal hasta /etc/passwd y el servidor devuelve su contenido en la respuesta HTTP.

La lección central de este laboratorio es que dos componentes del sistema (la capa que valida y la capa que finalmente abre el archivo) pueden interpretar la misma cadena de forma distinta si uno de ellos trunca en un carácter especial que el otro no reconoce; esa discrepancia es exactamente el espacio que el byte nulo aprovecha.

8. Mitigación recomendada

- Rechazar explícitamente cualquier entrada que contenga bytes nulos (%00) u otros caracteres de control antes de realizar cualquier otra validación, en lugar de asumir que la cadena se comportará igual en todas las capas del sistema.
- Validar la ruta canonicalizada y resuelta del archivo (no solo el sufijo de la cadena cruda proporcionada por el usuario), confirmando que corresponda efectivamente a un archivo dentro del directorio permitido y con la extensión real esperada.
- Actualizar y mantener las plataformas y bibliotecas de manejo de archivos a versiones modernas, ya que la mayoría de los lenguajes y runtimes actuales ya rechazan bytes nulos incrustados en rutas de archivo por

defecto; el problema persiste principalmente en código legado o en integraciones directas con bibliotecas nativas.

- Preferir un identificador indirecto (ID numérico o nombre interno) traducido a la ruta real mediante un mapeo controlado del servidor, evitando construir rutas directamente a partir de la entrada del usuario.
- Validar la entrada contra una lista blanca (allow-list) de nombres o extensiones permitidos, en lugar de una comprobación de sufijo sobre una cadena que puede contener caracteres inesperados.
- Aplicar el principio de menor privilegio: ejecutar el proceso del servidor con un usuario restringido sin permisos de lectura sobre archivos sensibles del sistema operativo.
- Registrar y monitorear solicitudes cuyo parámetro de archivo contenga secuencias de codificación sospechosas (%00 y otros caracteres de control codificados), ya que son un fuerte indicador de intentos de bypass.